

POLITECNICO
MILANO 1863

Reti Logiche

Dispositivi Programmabili

Logiche programmabili

- Sono dispositivi hardware che mettono a disposizione componenti logici più o meno complessi che possono essere connessi tra loro (programmazione delle connessioni) a seconda delle esigenze di progetto
- Dispongono di:
  - Componenti logici (Porte logiche, Flip-flop, Buffer…)
  - Linee di connessione
- Tipologie di dispositivi programmabili :
  - dispositivi logici programmabili a 2 livelli
    - ROM (Read-Only Memory),
    - PLA (Programmable Logic Array)
    - PAL (Programmable Array Logic)
  - CPLD
  - FPGA

Modalità di programmazione

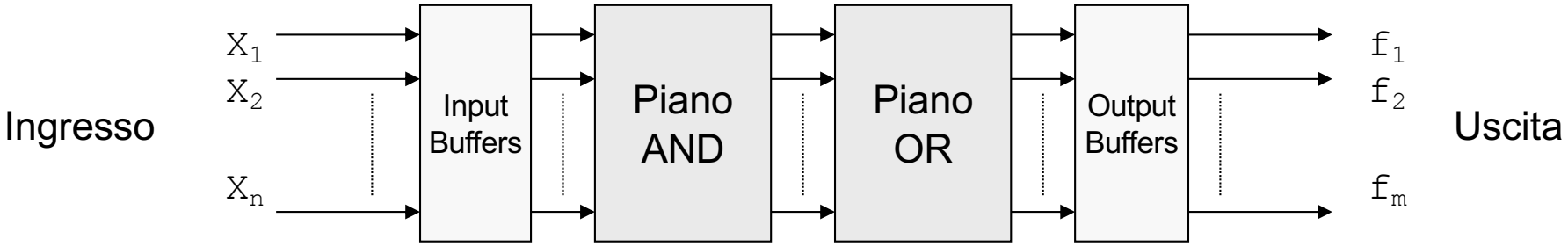
- Programmabili una sola volta (*One-Time Programmable* - OTP)
  - durante la fase non attiva del dispositivo (Fuse/antifuse)
- Riprogrammabili (*Reprogrammable*)
  - durante la fase non attiva del dispositivo (E^2PROM, SRAM-based)
- Riconfigurabili (*Reconfigurable*)
  - durante la fase attiva del dispositivo (SRAM-based)

Logiche programmabili a 2 livelli

- Funzioni a 2 livelli del tipo SOP a n ingressi e m uscite

$$f_i = f_i(x_1, x_2, \ldots, x_n) \text{ con } i=\{1, 2, \ldots, m\}$$

- Dispongono di:
  - Un numero di ingressi fissato;
  - Un numero di uscite fissato;
  - Un piano AND, per la costruzione dei mintermini o degli implicanti;
  - Un piano OR, per la somma dei mintermini o degli implicanti;
  - Buffer di ingresso e di uscita (per ragioni elettriche e funzionali)

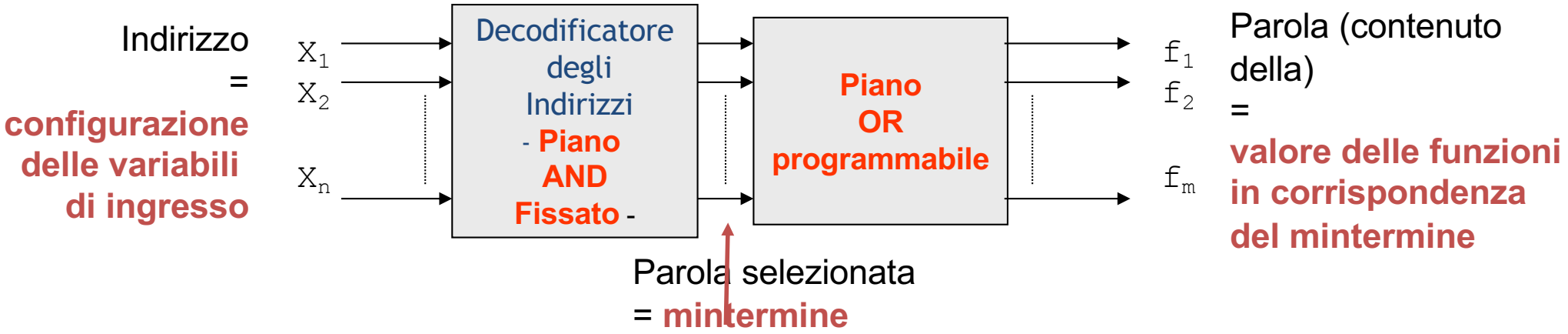


Logiche programmabili a 2 livelli

- Read-Only Memory (ROM)
  - Piano AND fissato.
    - Implementa **tutti** i possibili mintermini (decoder).
  - Piano di OR adattabile.
- Programmable Logic Array (PLA)
  - Piano AND programmabile.
    - E' dato il numero di termini prodotto generabili, si programmano solo i mintermini/implicanti necessari.
  - Piano OR programmabile.
- Programmable Array Logic (PAL)
  - Piano AND programmabile.
    - E' dato il numero di termini prodotto generabili, si programmano solo i mintermini/implicanti necessari.
  - Piano di OR fissato.
    - Ogni funzione (OR) può essere costruita solo con un certo sottoinsieme (cablato) di termini prodotto.

Read-Only Memory (ROM)

- Un Memoria a Sola Lettura (ROM) implementa la prima forma canonica di m funzioni di uscita a n ingressi
  - Somma di Prodotti (SOP)
- In una ROM, una configurazione di ingresso, denominata *indirizzo,* è associata una configurazione di uscita, denominata *parola*



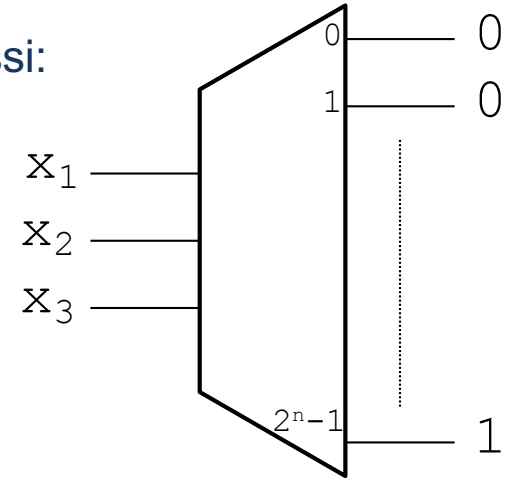
METODO DI SINTESI: SELEZIONI I MINTERMINI DI CUI HO BISOGNO!

ROM: decodificatore degli indirizzi

- Decodificatore degli indirizzi (*Address decoder*)
  - Il decodificatore degli indirizzi nelle memorie ROM realizza tutti i 2^n mintermini, dove n sono le variabili di ingresso x_i
- Infatti
  - Gli ingressi sono le variabili x_i
  - Una ed una sola uscita è attiva alla volta; le uscite del decoder sono tutti i mintermini costruiti a partire dalle variabili di ingresso
    - Esempio di decodificatore a 3 ingressi:

Esempio:

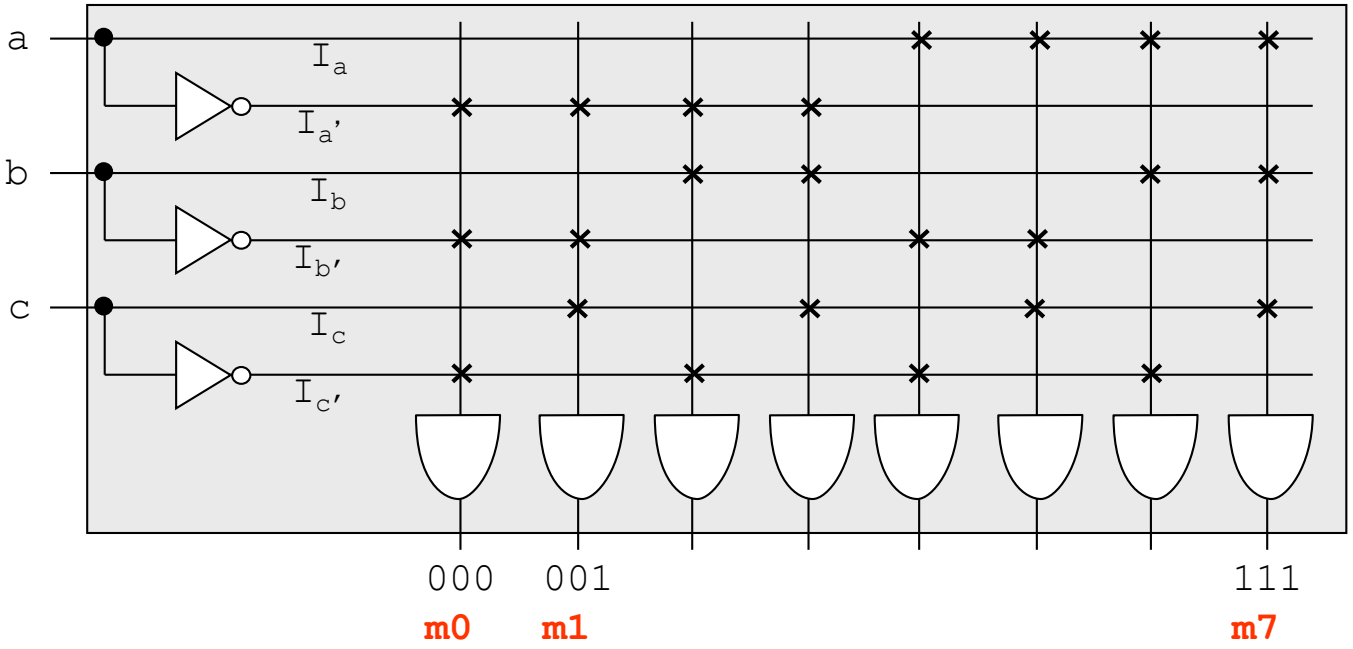
$x_1x_2x_3 = 111$

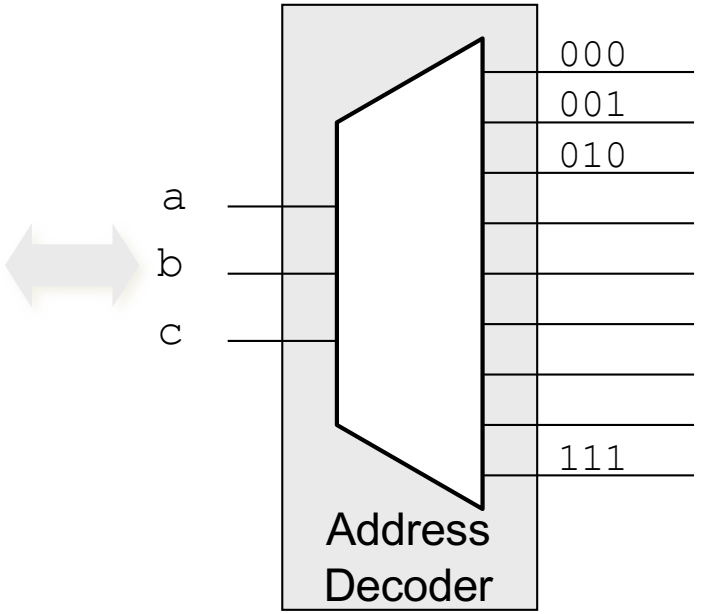


ROM: *piano AND e decodificatore degli indirizzi*

- Schema logico del piano di AND che implementa l'*Address Decoder*
  - Per semplicità si utilizza la rappresentazione che si riferisce al *decoder*.

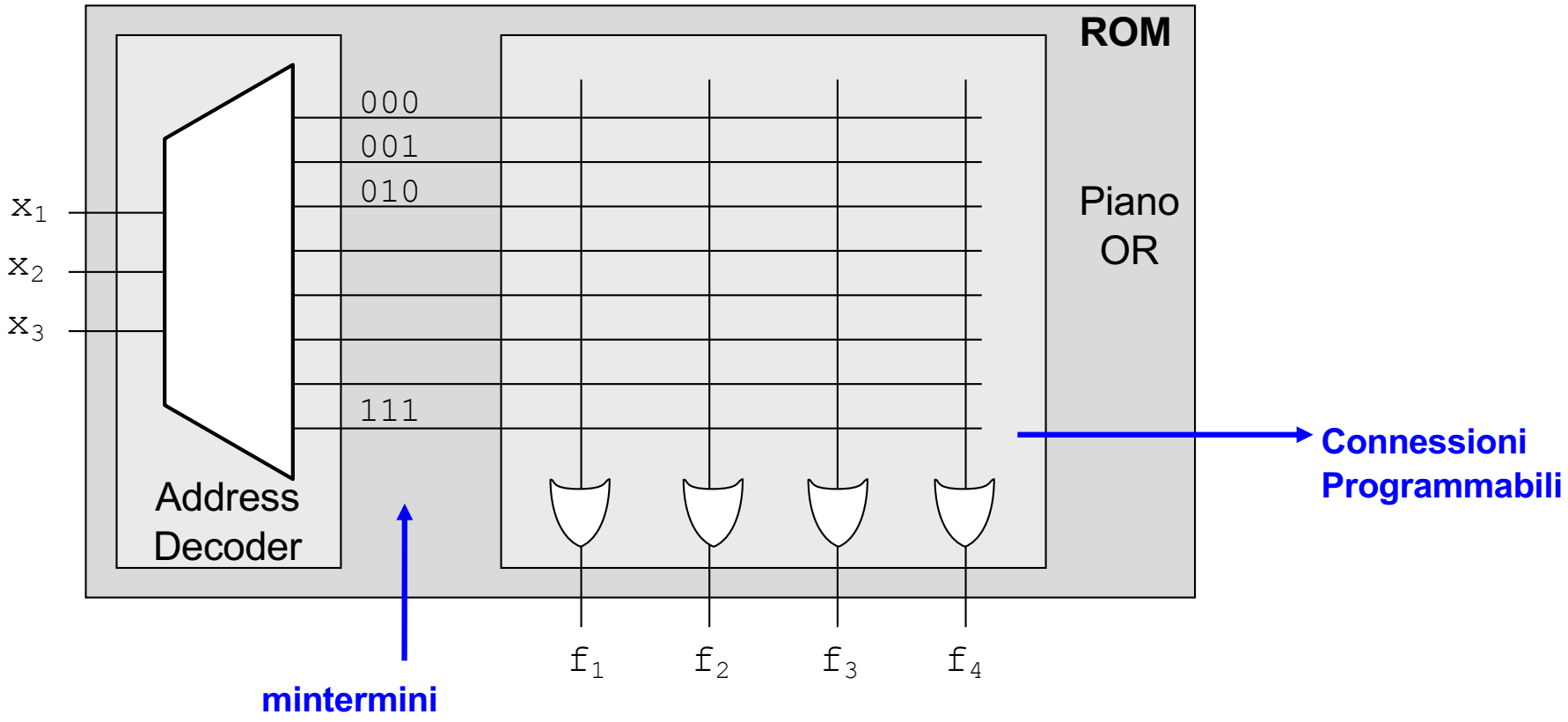
Significato: ogni croce indica quali variabili sono coinvolte, il tipo di porta indica schematicamente come tali variabili collegate tra loro.





ROM: *schema logico*

- Schema logico di una ROM:
  - Esempio di una ROM a 3 ingressi e 4 uscite (non programmata).

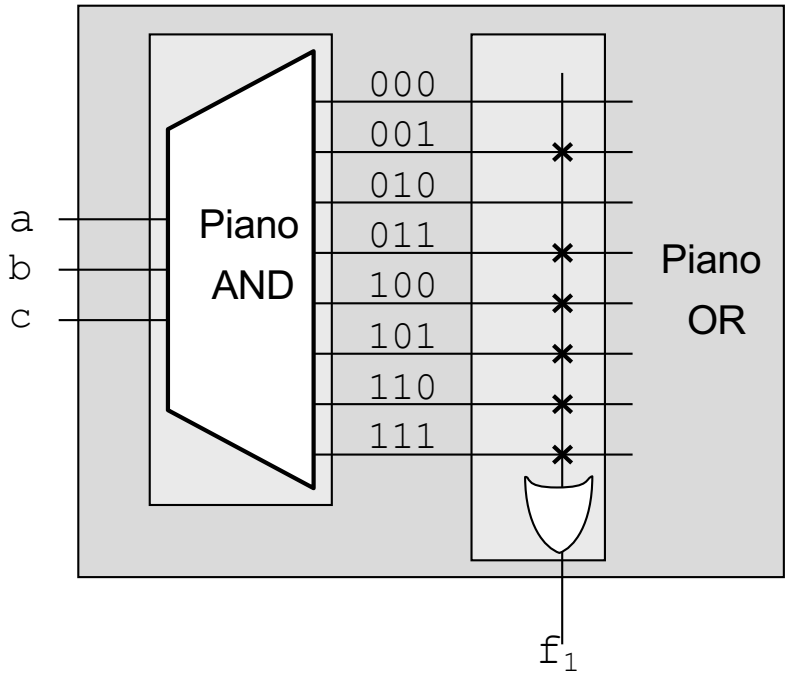


ROM: *Esempio 1*

- Si realizzi usando una ROM la funzione $f_1 = a + c$
  1. Primo passo: portarla in forma canonica
     - $f_1 = a'b'c+a'bc+ab'c'+ab'c+abc'+abc$
  2. Secondo passo: costruire la tabella delle verità
  3. Terzo passo selezionare i mintermini necessari per f_1

$f_1 = a'b'c + a'bc + ab'c' + ab'c + abc' + abc$

abc	f_1
000	0
001	1
010	0
011	1
100	1
101	1
110	1
111	1



ROM: *Esempio 2*

- Si realizzi usando una ROM la funzione a più uscite che segue:

$$f_1 = ab + ac' + a'b'c$$
$$f_2 = ab + ac + a'b'c$$

Soluzione:

1. Prima forma canonica della funzione a più uscite:
2. Tabella della verità della funzione a più uscite:

$$f_1 = abc + abc' + ab'c' + a'b'c$$
$$f_2 = abc + abc' + ab'c + a'b'c$$

abc	f_1	f_2
000	0	0
001	1	1
010	0	0
011	0	0
100	1	0
101	0	1
110	1	1
111	1	1

ROM: *Esempio 2*

- Si realizzi usando una ROM la funzione a più uscite che segue:

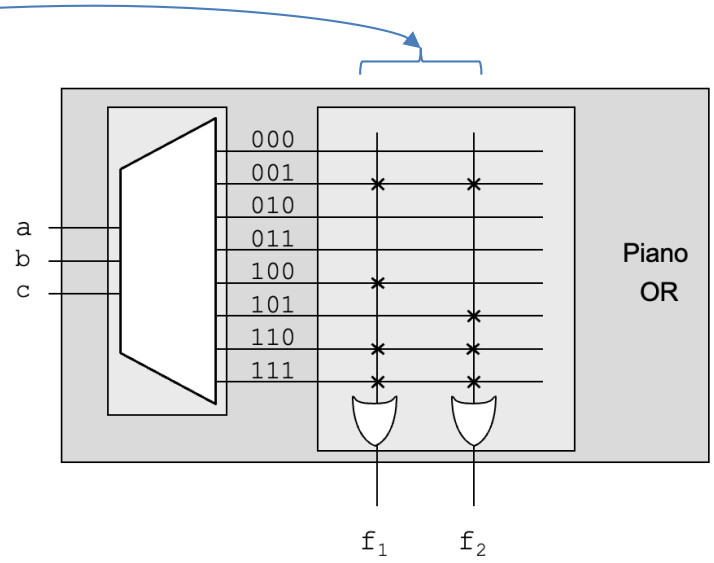
$$f_1 = ab + ac' + a'b'c$$
$$f_2 = ab + ac + a'b'c$$

Soluzione:

1. Prima forma canonica della funzione a più uscite:
2. Tabella della verità della funzione a più uscite:

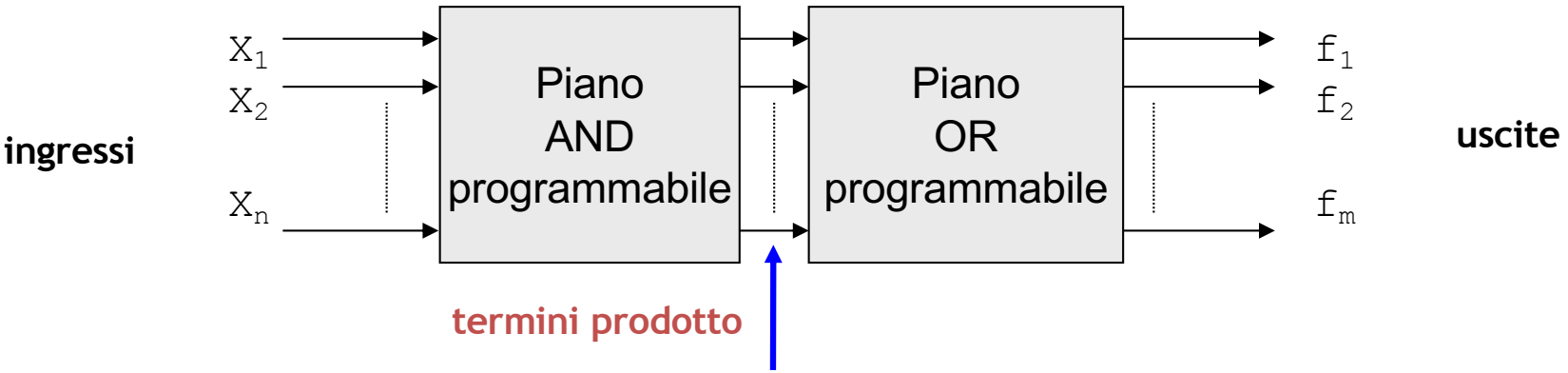
$$f_1 = abc + abc' + ab'c' + a'b'c$$
$$f_2 = abc + abc' + ab'c + a'b'c$$

abc	f_1	f_2
000	0	0
001	1	1
010	0	0
011	0	0
100	1	0
101	0	1
110	1	1
111	1	1



Programmable Logic Array (PLA)

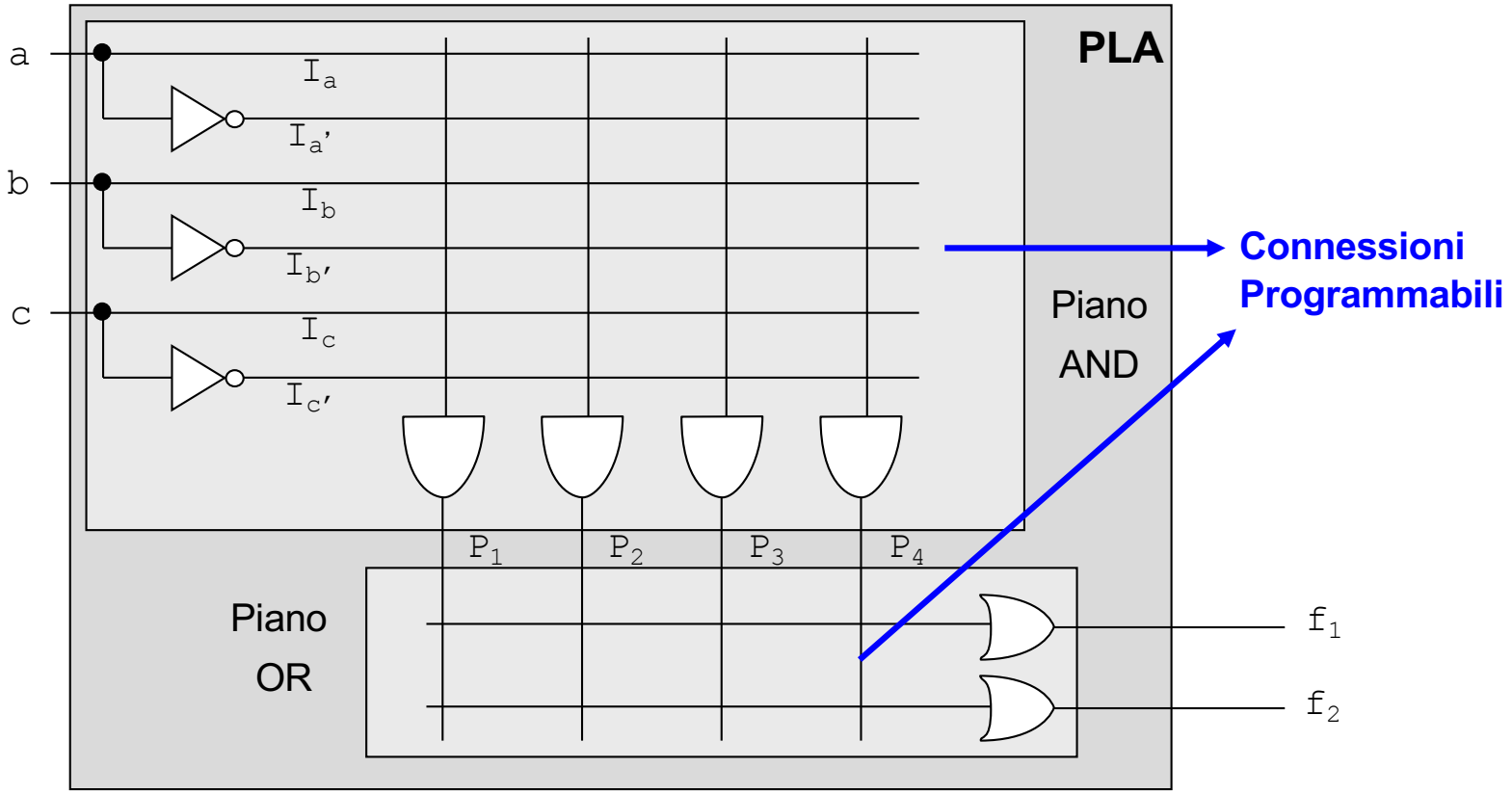
- Un Array Logico Programmabile (*Programmable Logic Array* - PLA) consente di implementare una somma di prodotti espressa in forma minima a due livelli (*somma di implicanti*)
  - Nota: l'estensione a forme non minime è naturale.
- In generale una PLA è definita da: numero di ingressi (n° variabili delle funzioni), numero dei termini prodotto generabili, numero di uscite (n° di funzioni realizzabili)



METODO DI SINTESI: GENERARE IL MINOR NUMERO DI IMPLICANTI DA SELEZIONARE NEL PIANO OR!

Programmable Logic Array (PLA): *schema logico*

- Schema logico di una PLA
  - Esempio di PLA a 3 ingressi, 2 uscite, 4 termini prodotto (non programmata):



PLA: *Esempio*

- Esempio :
  - Realizzazione delle funzioni:

$f_1 = ab + ac' + a'b'c$

$f_2 = ab + ac + a'b'c$

- Prodotti:

$P_1 = ab$

$P_2 = ac$

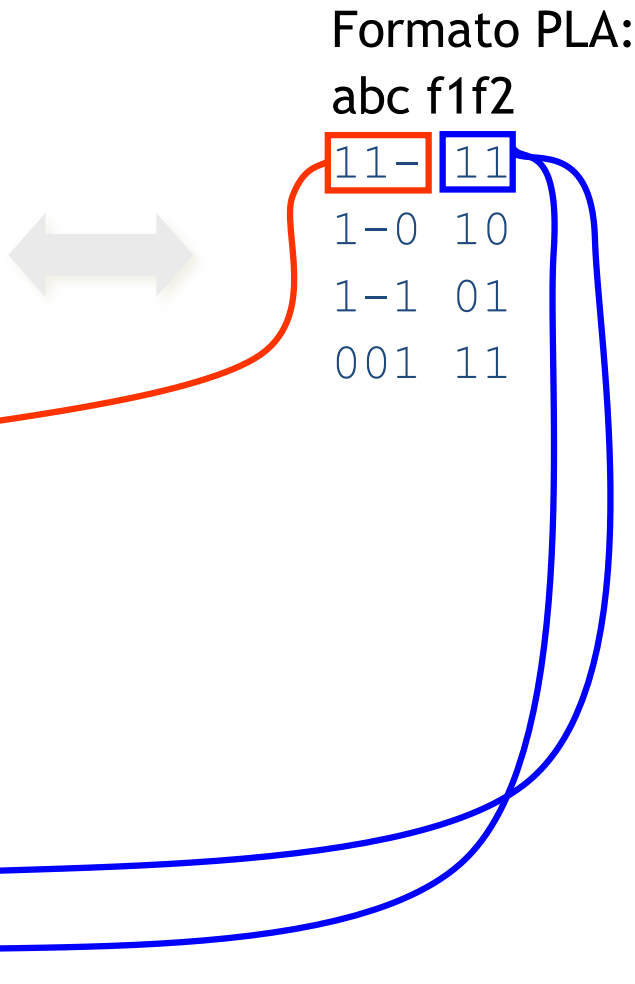
$P_3 = ac'$

$P_4 = a'b'c$

- Somme:

$f_1 = P_1 + P_3 + P_4$

$f_2 = P_1 + P_2 + P_4$

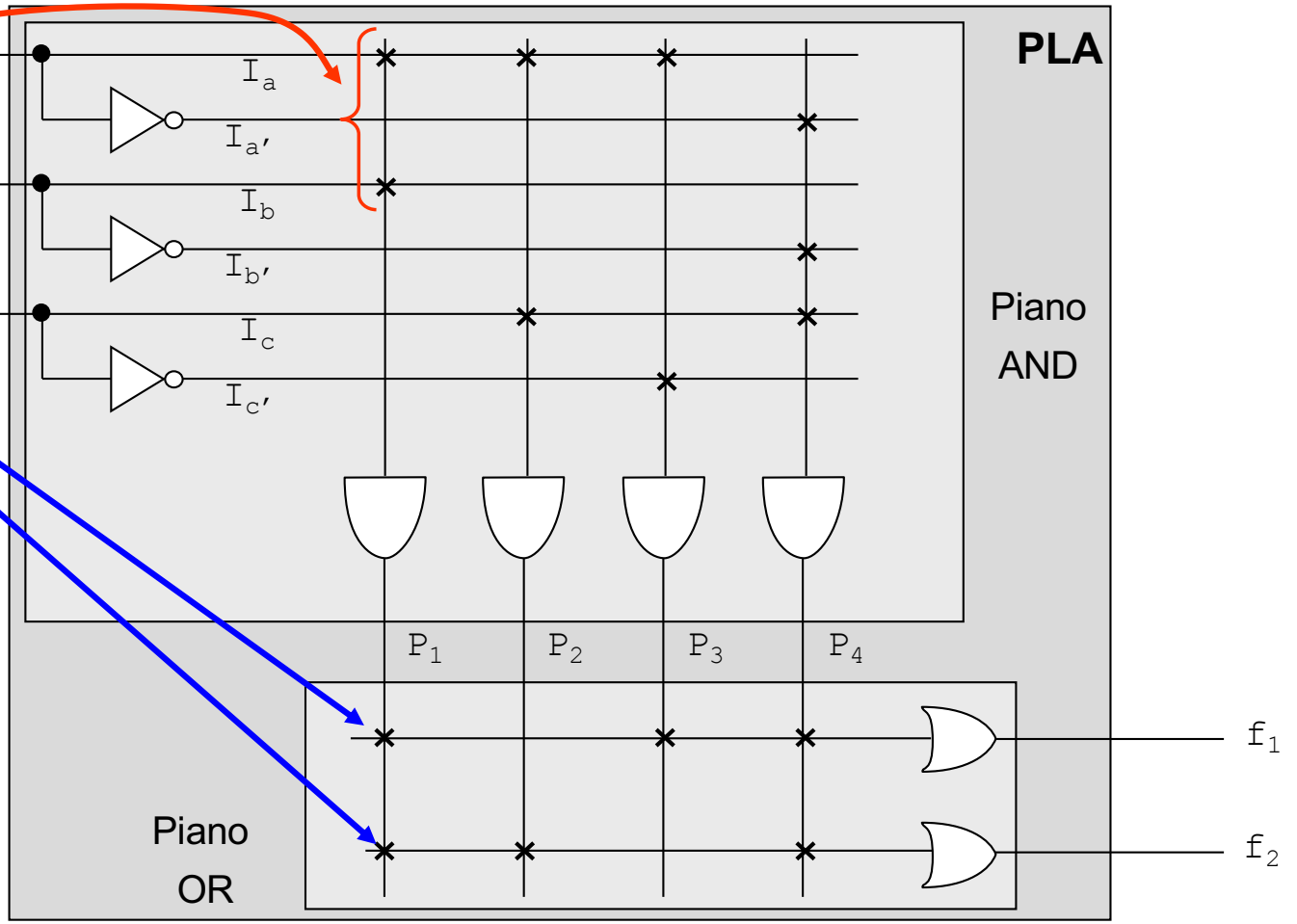


PLA: *Esempio*

- Esempio (cont.): (PLA programmata per le funzioni f_1 ed f_2)

Formato PLA:

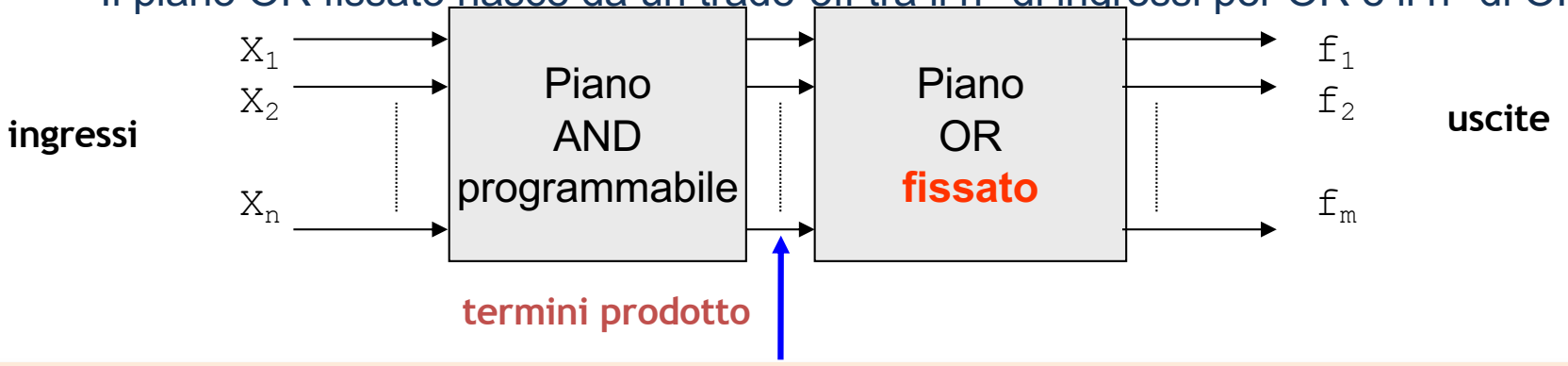
```
11- 11
1-0 10
1-1 01
001 11
```



Programmable Array Logic (PAL)

- Un Array Programmabile Logico (PAL) consente di implementare una somma di prodotti espressa in forma minima a due livelli (*somma di implicanti*)
  - PLA e PAL coprono lo stesso spazio d'applicazione.

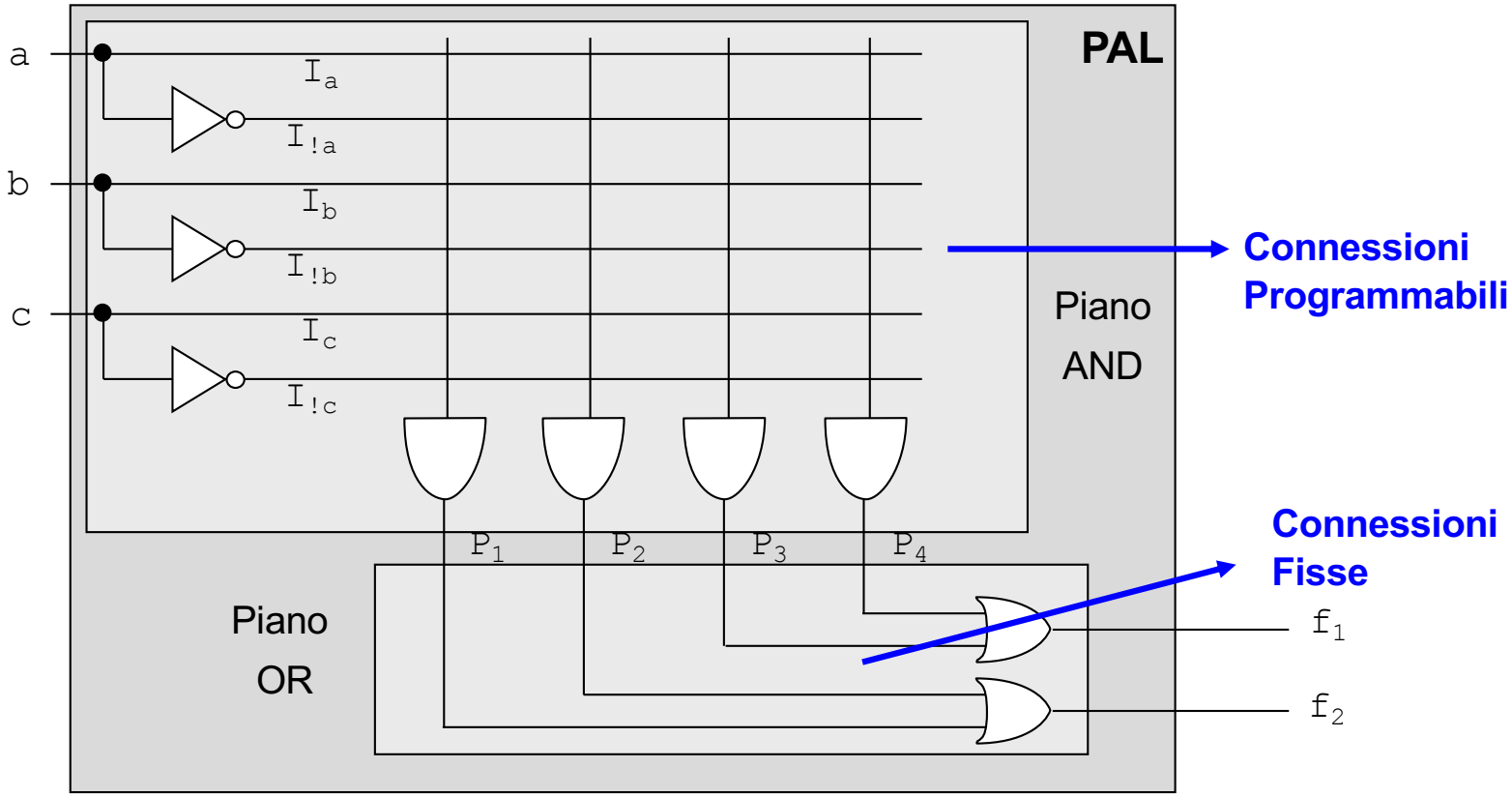
- In generale una PAL è definita da: numero di ingressi (n° variabili delle funzioni), numero dei termini prodotto generabili, numero di uscite (n° di funzioni realizzabili)
  - Il piano OR fissato nasce da un trade-off tra il n° di ingressi per OR e il n° di OR



METODO DI SINTESI: GENERARE IL MINOR NUMERO DI IMPLICANTI DA SELEZIONARE **SECONDO** IL PIANO DEGLI OR

PAL: *schema logico*

- Schema logico di una PAL
  - Esempio di PAL a 3 ingressi, 4 termini prodotto, 2 uscite (non programmata):



PLA e PAL

- Nelle **PLA**, le uscite possono condividere termini prodotto
- Nelle **PAL**, le uscite non possono condividere termini prodotto e inoltre il numero di ingressi alle porte OR è fissato
  - Il piano OR fissato può implicare comunque una realizzazione multilivello
- A pari funzionalità da implementare,
  - la sezione AND di una PAL deve essere più grande (termini prodotto replicati) di quella di una PLA
  - Le PLA sono più lente delle PAL a causa della programmabilità della sezione OR

PAL: *Esempio 1*

- Realizzazione tramite una PAL (a 3 ingressi, 6 termini prodotto, OR a 3 ingressi e 2 uscite) delle funzioni:

$f_1 = ab + ac' + a'b'c$

$f_2 = ab + ac + a'b'c$

- Prodotti:

$P_1 = ab$

$P_2 = ac'$

$P_3 = a'b'c$

$P_4 = ab$

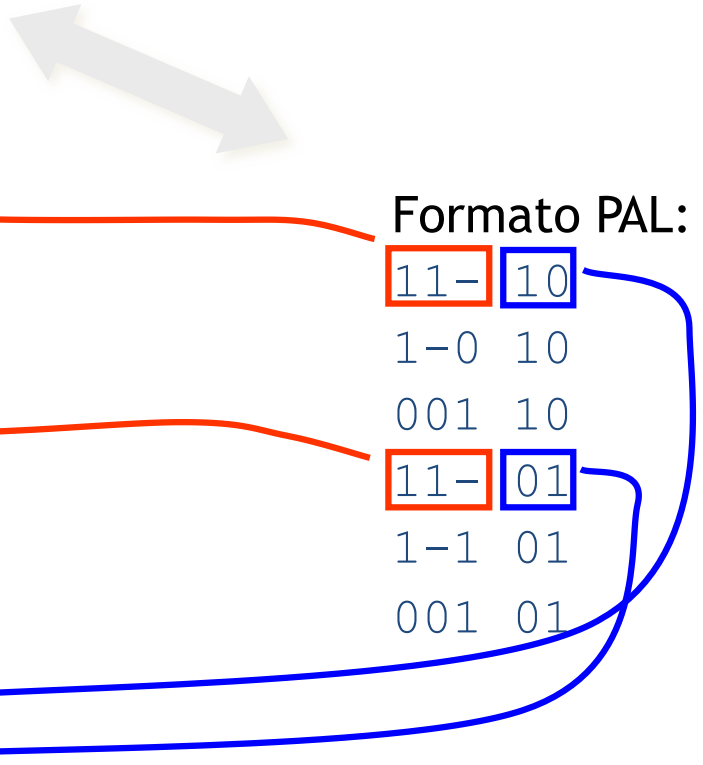
$P_5 = ac$

$P_6 = a'b'c$

- Somme:

$f_1 = P_1 + P_2 + P_3$

$f_2 = P_4 + P_5 + P_6$

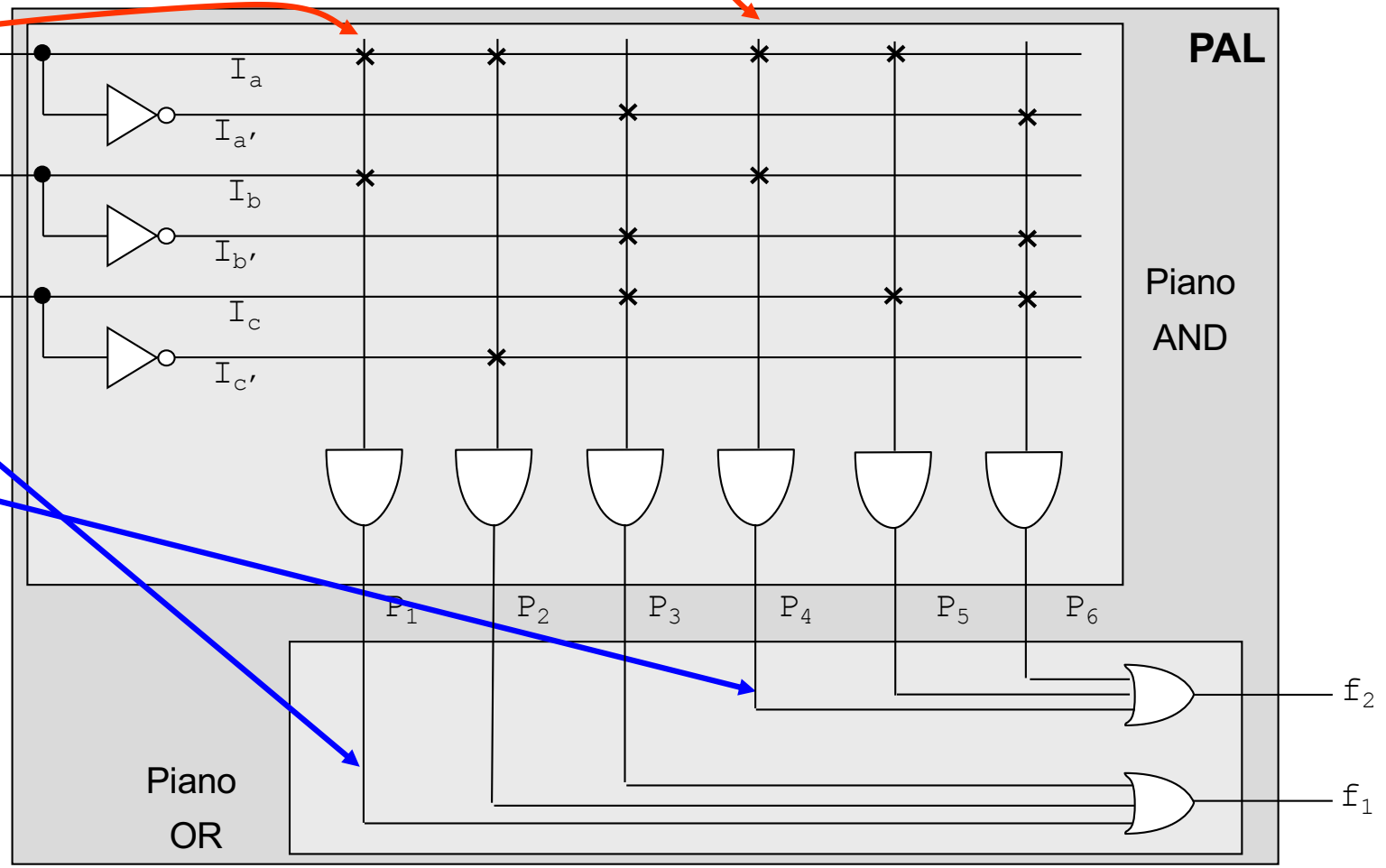


PAL: *Esempio 1*

- Esempio 1 (cont.): (PAL programmata per le funzioni f_1 ed f_2)

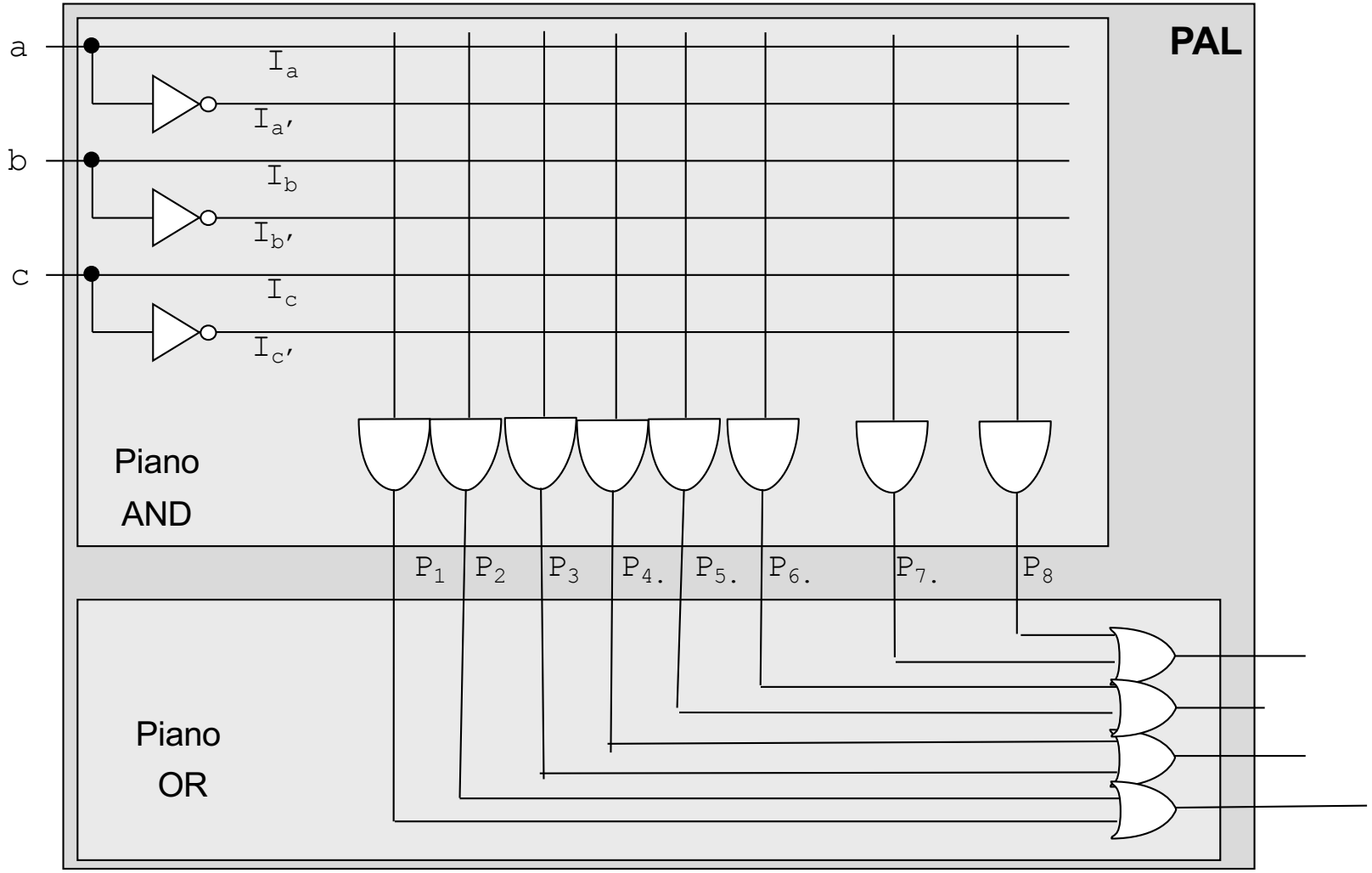
Formato PAL:

```
11- 10
1-0 10
001 10
11- 01
1-1 01
001 01
```



PAL: *Esempio BIS*

- E se fossero disponibili solo OR a 2 ingressi?

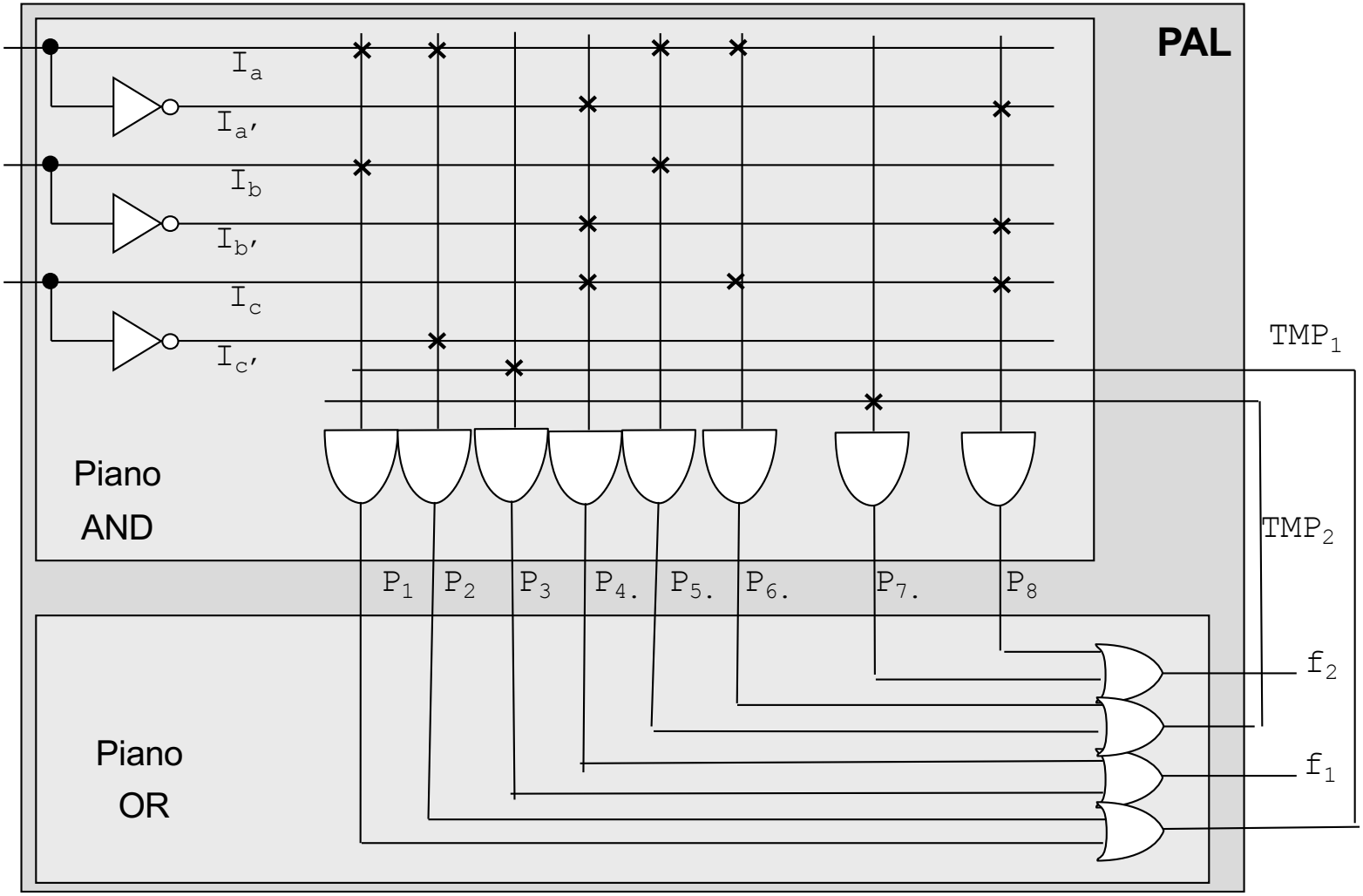


PAL: *Esempio BIS*

E se fossero disponibili solo OR a 2 ingressi?

- Prodotti:
  - $P_1 = ab$
  - $P_2 = ac'$
  - $P_3 = tmp_1$
  - $P_4 = a'b'c$
  - $P_5 = ab$
  - $P_6 = ac$
  - $P_7 = tmp_2$
  - $P_8 = a'b'c$
- Somme:
  - $tmp_1 = P_1 + P_2$
  - $f_1 = P_4 + P_3$
  - $tmp_2 = P_6 + P_5$
  - $f_2 = P_7 + P_8$

Necessaria PAL con 5 ingressi primari 4 uscite e 8 prodotti!

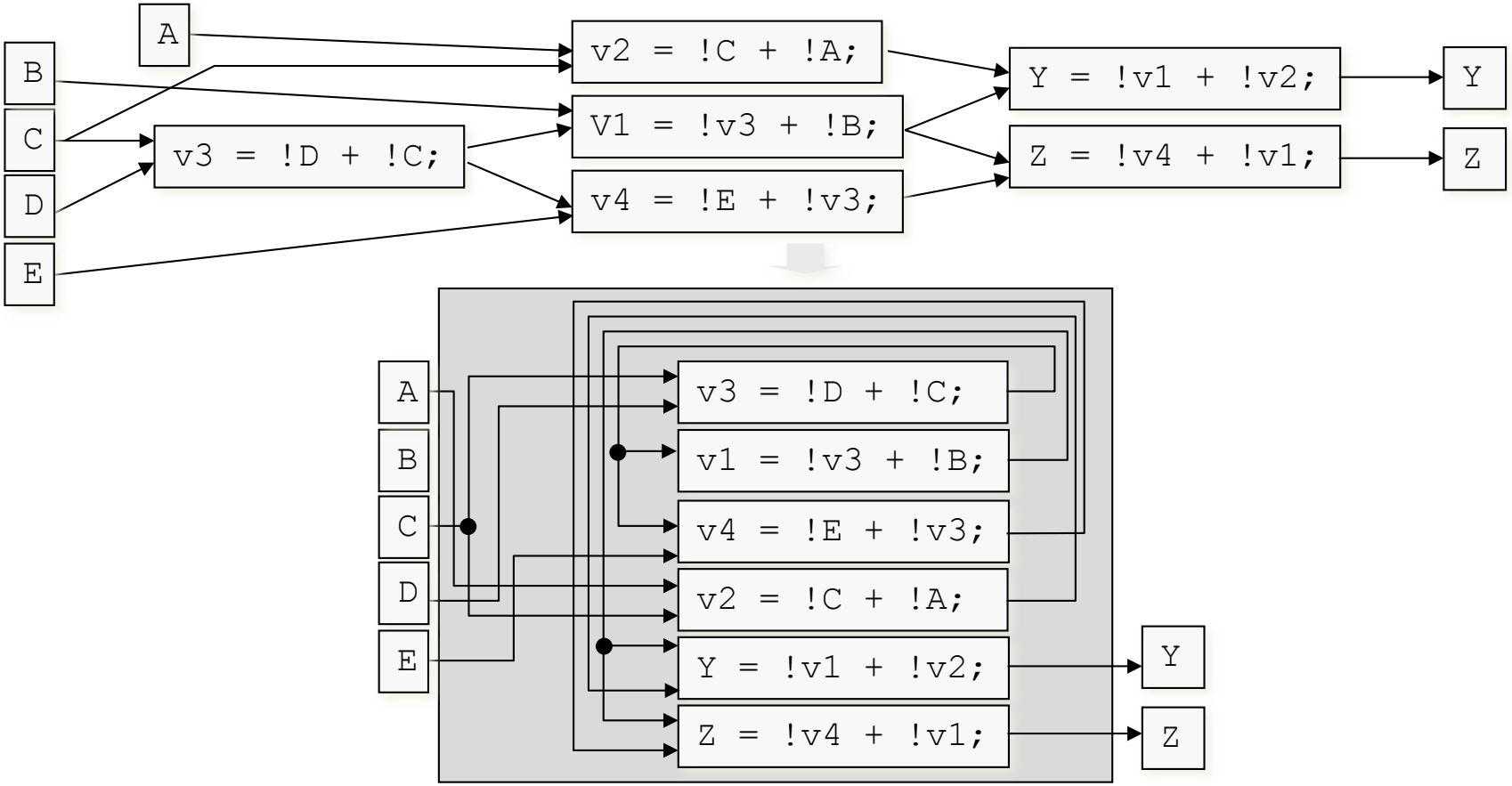


PLA e PAL Avanzate

- Lo schema base mostrato consente di realizzare solo reti combinatorie a due livelli
- Questo limite è superato:
  - Introducendo delle linee di retroazione
    - Permette di implementare reti combinatorie multi livello a più uscite
  - Introducendo elementi di memoria (bistabili)
    - Permette di implementare macchine sequenziali sincrone in cui la parte combinatoria è costituita da una rete multi livello a più uscite

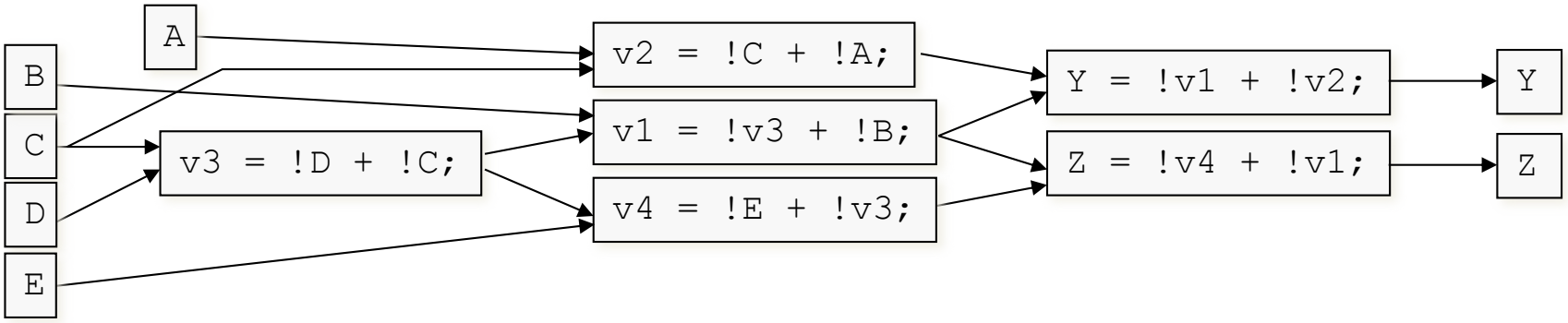
PLA avanzate: rete multi livello - esempio

- Esempio di implementazione tramite PLA di una rete combinatoria multi livello a più uscite.



PLA: rete multi livello - esempio

- Esempio di implementazione di una rete combinatoria multi livello a più uscite.



Funzioni da realizzare (piano OR)

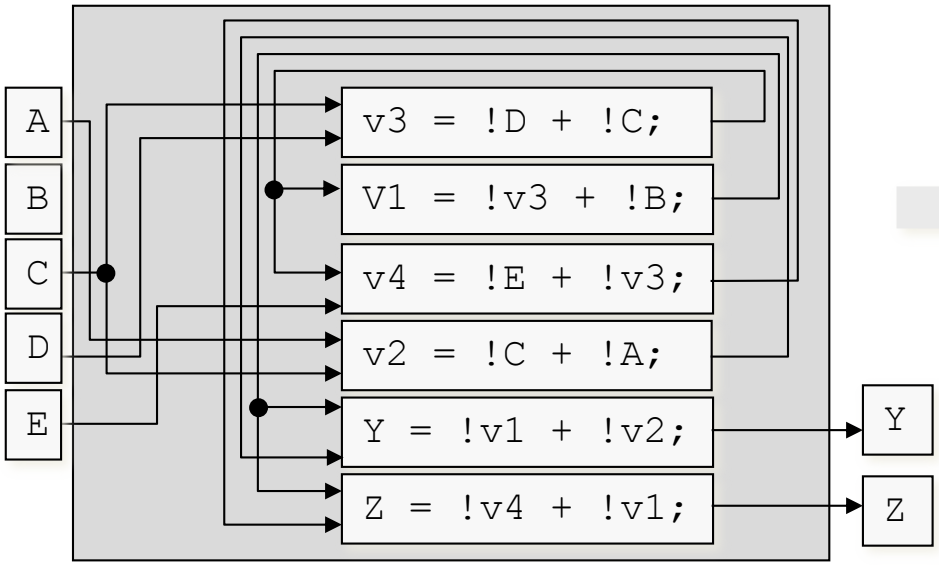
```
v1 = !v3 + !B
v2 = !C + !A
v3 = !D + !C
v4 = !E + !v3
Y = !v1 + !v2     uscita
Z = !v4 + !v1     uscita
```

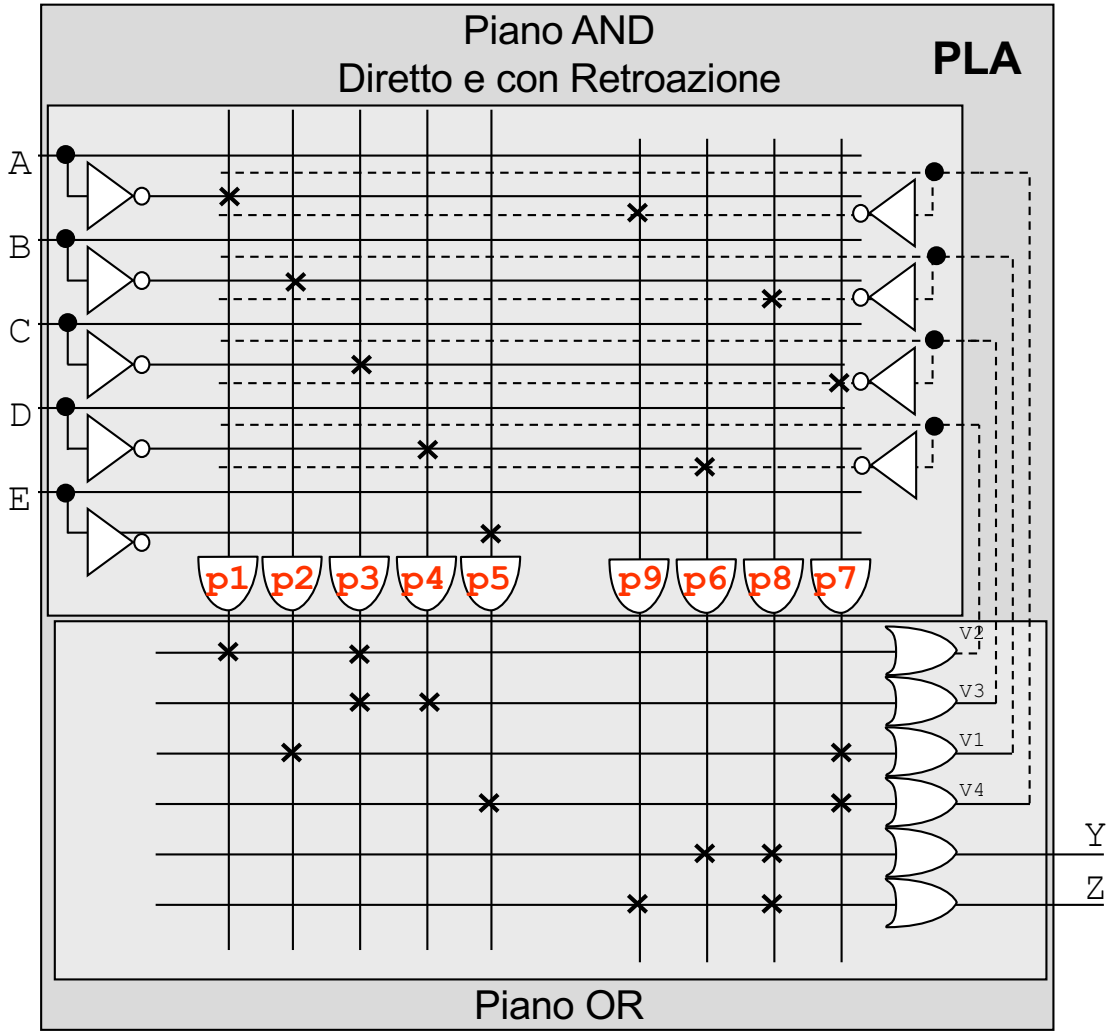
Termini prodotto da realizzare (piano AND)

```
p1 = !A       p6 = !v1
p2 = !B       p7 = !v2
p3 = !C       p8 = !v3
p4 = !D       p9 = !v4
p5 = !E
```

PLA: rete multi livello - esempio

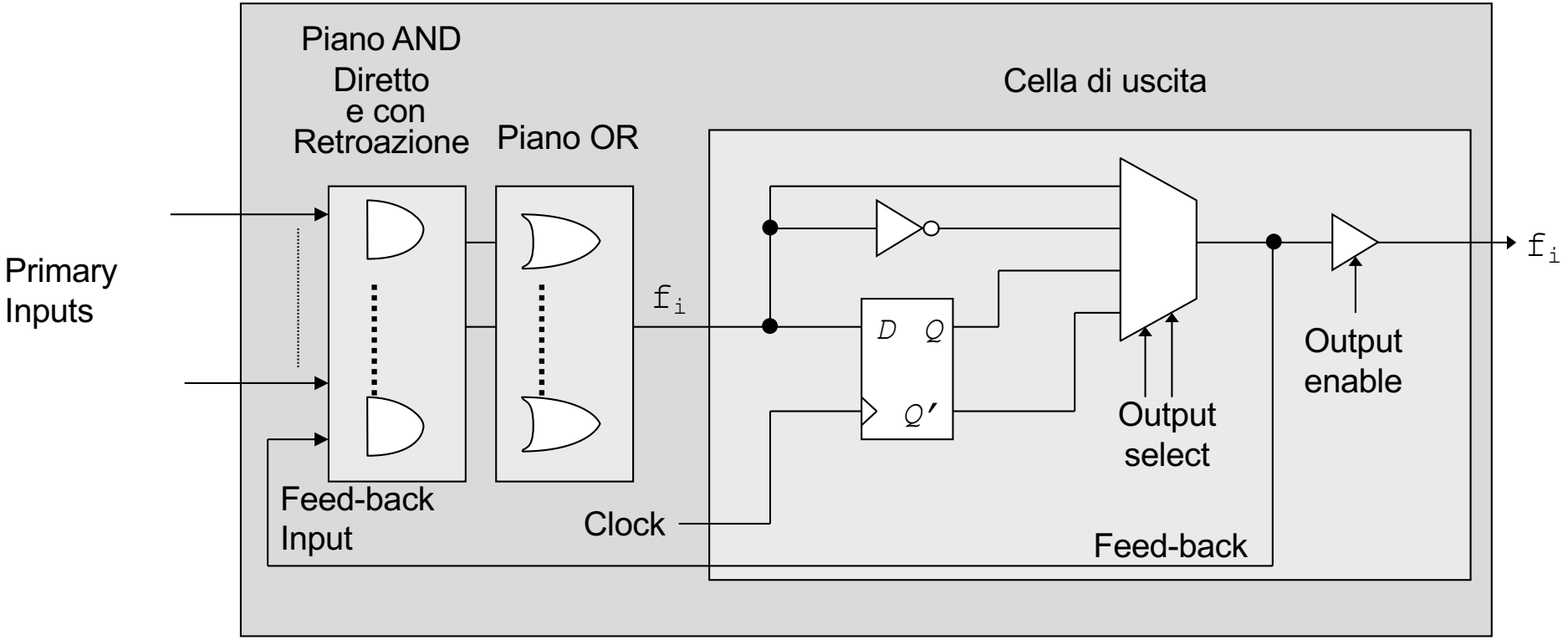
- Esempio (cont):





PLA e PAL Avanzate: struttura generale

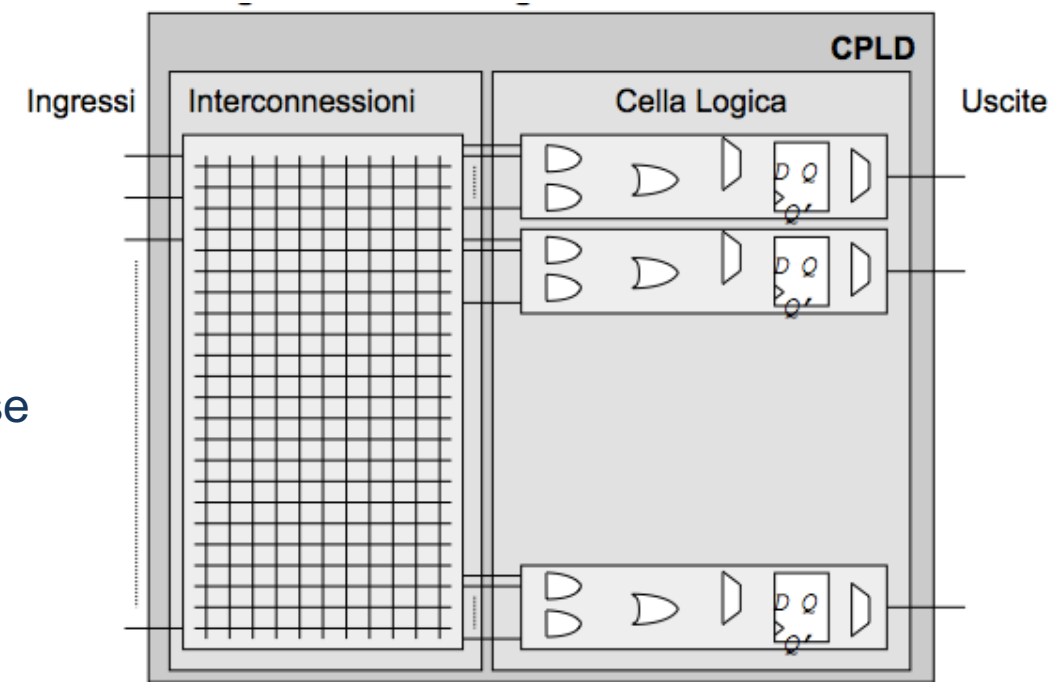
- Struttura logica dei dispositivi avanzati basati su PLA a PAL con retroazione e bistabili



Complex Programmable Logic Devices

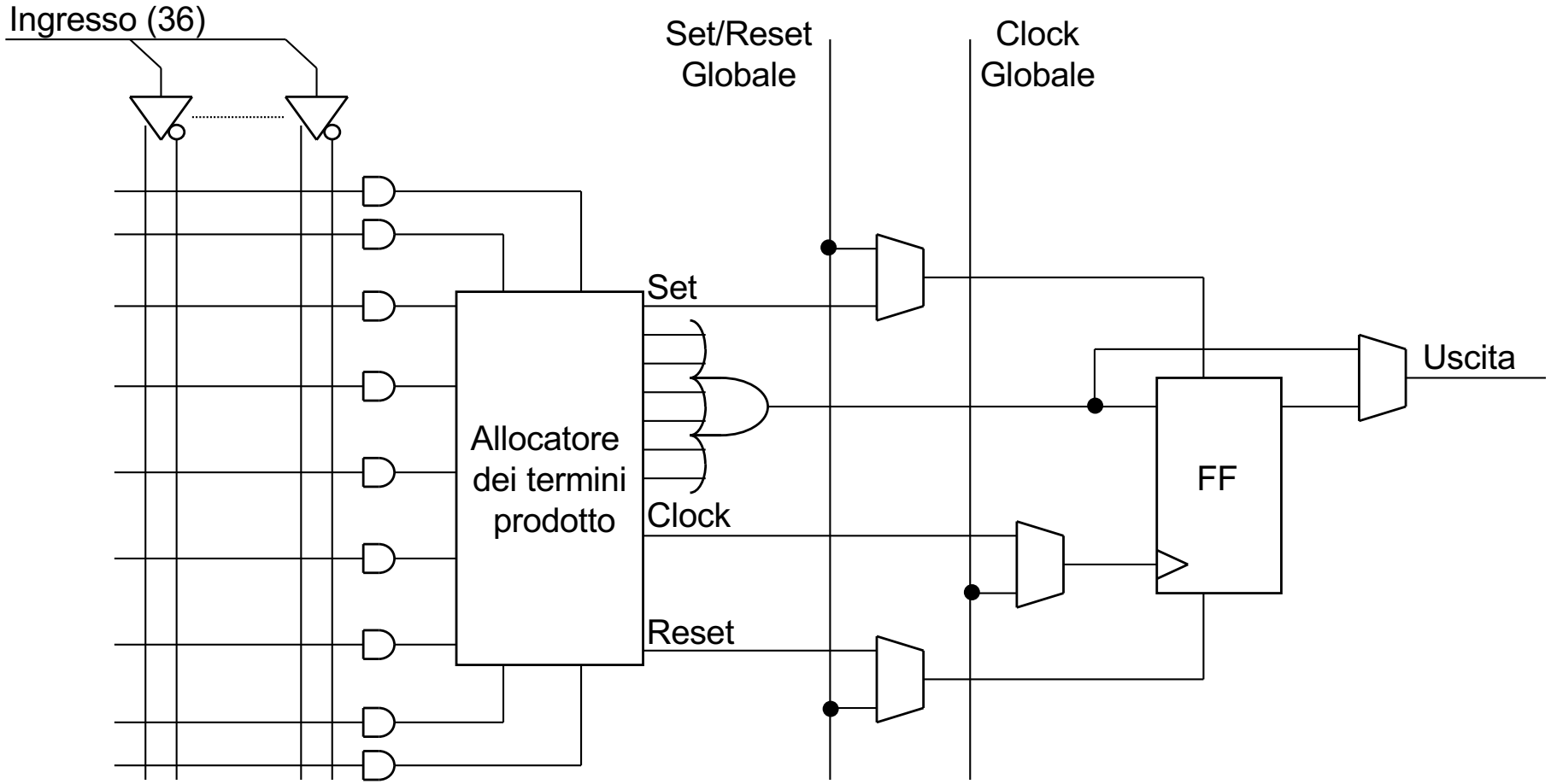
- I Dispositivi Logici Programmabili Complessi (*Complex Programmable Logic Devices - CPLD*) sono una evoluzione delle PLA e PAL

- Sono caratterizzati da:
  - Connessioni globali
  - Logica concentrata
- Rispetto a PAL e PLA:
  - Hanno dimensioni molto maggiori
  - Sono costituite da celle più complesse
  - Consentono di ottenere prestazioni più elevate
  - Hanno una struttura più regolare e facilmente programmabile.



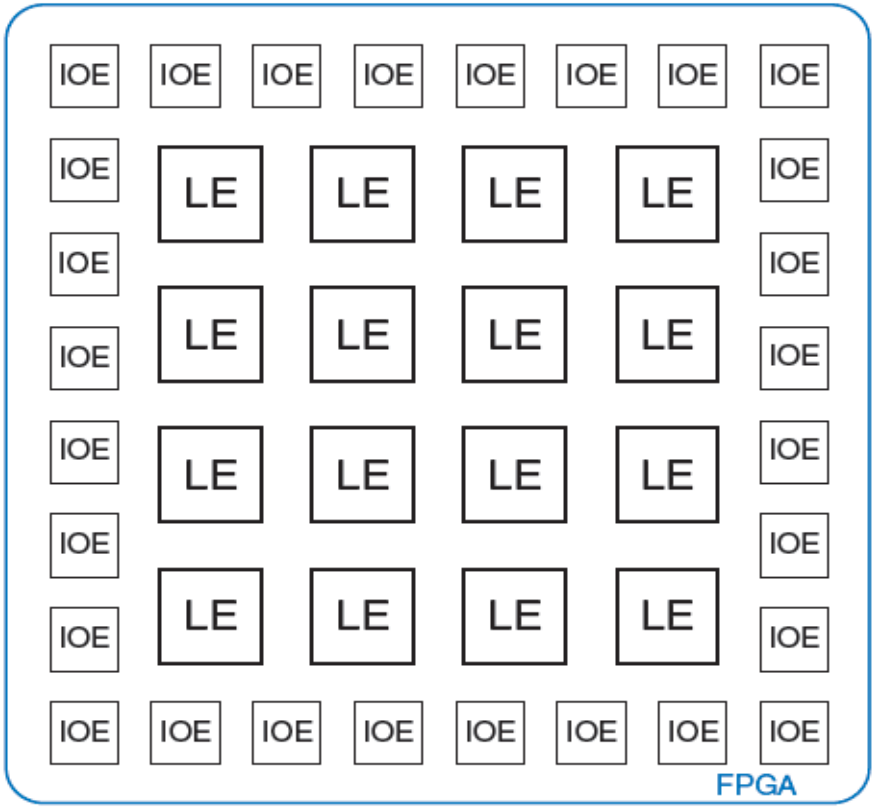
Complex Programmable Logic Devices

- Esempio: Cella logica della *Xilinx XC9500*



General FPGA Layout

- Composed of:
  - **LEs** (Logic elements): perform logic
  - **IOEs** (Input/output elements): interface with outside world
  - **Programmable interconnection:** connect LEs and IOEs
  - Some FPGAs include other building blocks such as multipliers and RAMs



LE: Logic Element

- Composed of:
  - **LUTs** (lookup tables): perform combinational logic
  - **Flip-flops:** perform sequential logic
  - **Multiplexers:** connect LUTs and flip-flops

e.g. Altera (INTEL) Cyclone IV LE

